

FD02-Informe-Vision



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe de Visión de Producto

Sistema Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)

Curso: Calidad y Pruebas de Software

Docente: Patrick Cuadros Quiroga

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

Sistema *Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)*

Informe de Visión de Producto

Versión 1.3

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-04	Primera versión del documento
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-04	Ampliación de detalle académico en FD02
1.2	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-05	Correcciones menores y ajustes de formato
1.3	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Unificación del formato institucional

ÍNDICE GENERAL

1. Introduccion

- 1.1 Proposito
- 1.2 Alcance
- 1.3 Definiciones, Siglas y Abreviaturas
- 1.4 Referencias
- 1.5 Vision General

2. Posicionamiento

- 2.1 Oportunidad de negocio
- 2.2 Definicion del problema

3. Descripcion de los interesados y usuarios

- 3.1 Resumen de los interesados
- 3.2 Resumen de los usuarios
- 3.3 Entorno de usuario
- 3.4 Perfiles de los interesados
- 3.5 Perfiles de los usuarios
- 3.6 Necesidades de los interesados y usuarios

4. Vista General del Producto

- 4.1 Perspectiva del producto
- 4.2 Resumen de capacidades
- 4.3 Suposiciones y dependencias
- 4.4 Costos y precios
- 4.5 Licenciamiento e instalacion

5. Caracteristicas del producto

6. Restricciones

7. Rangos de calidad

8. Precedencia y Prioridad

9. Otros requerimientos del producto

- 9.1 Estandares legales
- 9.2 Estandares de comunicacion
- 9.3 Estandares de cumplimiento de la plataforma
- 9.4 Estandares de calidad y seguridad

CONCLUSIONES

RECOMENDACIONES

BIBLIOGRAFIA

WEBGRAFIA

Informe de Vision

1. Introduccion

1.1 Proposito

El presente informe define la vision funcional y academica del sistema *Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)*. Su proposito es establecer una referencia comun para estudiantes, docente evaluador y futuros mantenedores del proyecto, describiendo con claridad:

- El problema real que se busca resolver.
- El alcance de la solucion en su version actual.
- Las capacidades esperadas y sus limites.
- Los criterios de calidad y prioridad para su evolucion.

Este documento tambien cumple una funcion de trazabilidad, porque conecta el objetivo del proyecto con artefactos tecnicos verificables (CLI, modulos de analisis, reportes y pruebas).

1.2 Alcance

Alcance funcional (incluido):

- Analisis de proyectos Maven (`pom.xml`), Gradle Groovy/Kotlin (`build.gradle`, `build.gradle.kts`), npm (`package.json`) y Python (`pyproject.toml`, `requirements.txt`).
- Deteccion automatica del tipo de proyecto mediante el modulo `ProjectDetector`.
- Extraccion de dependencias y repositorios declarados en archivos de construccion.
- Consulta de versiones recientes por repositorio para identificar posibles desactualizaciones.
- Consulta de vulnerabilidades conocidas (CVE) utilizando OSS Index.
- Seleccion de fuente de vulnerabilidades con OSS Index (`--oss`), NIST NVD (`--nvd`) o modo automatico.
- Clasificacion de vulnerabilidades en directas y transitivas dentro del reporte.
- Emision de salida legible en consola y salida estructurada en JSON.
- Modo TUI interactivo para visualizacion y flujo de trabajo en terminal.
- Actualizacion guiada de dependencias con seleccion interactiva, confirmacion explicita y backup de archivos de build.

Alcance no funcional (incluido):

- Ejecucion local por linea de comandos en JVM.
- Compatibilidad operativa con Windows, Linux y macOS.
- Uso de bibliotecas de codigo abierto mantenidas por la comunidad.

Fuera de alcance (version actual):

- Interfaz grafica web o de escritorio.
- Remediacion automatica de dependencias en archivos fuente.
- Integracion nativa con plataformas empresariales propietarias.
- Sustitucion completa de herramientas SCA comerciales.

1.3 Definiciones, Siglas y Abreviaturas

- **CLI (Command Line Interface):** interfaz de linea de comandos para ejecutar funciones del sistema.
- **CVE (Common Vulnerabilities and Exposures):** identificador estandar de vulnerabilidades.
- **SCA (Software Composition Analysis):** analisis de componentes de terceros y sus riesgos.
- **OSS Index:** servicio de Sonatype para consulta de vulnerabilidades en dependencias.
- **JVM (Java Virtual Machine):** plataforma de ejecucion para aplicaciones Java/Kotlin.
- **JSON (JavaScript Object Notation):** formato de salida estructurada para integracion automatizada.
- **Dependencia transitiva:** libreria incluida indirectamente por otra dependencia directa.
- **Coordenadas de dependencia:** forma groupId:artifactId:version para identificar componentes.

1.4 Referencias

- Documento base de uso: README.md.
- Configuracion de build: build.gradle.kts.
- Punto de entrada CLI:
src/main/kotlin/com/depanalyzer/cli/DepAnalyzerCli.kt.
- Motor de analisis:
src/main/kotlin/com/depanalyzer/core/ProjectAnalyzer.kt.
- Modelo de reporte:
src/main/kotlin/com/depanalyzer/report/DependencyReport.kt.
- Generador de reportes:
src/main/kotlin/com/depanalyzer/report/ReportGenerator.kt.
- Cliente de vulnerabilidades:
src/main/kotlin/com/depanalyzer/repository/OssIndexClient.kt.
- Cliente de enriquecimiento NVD:
src/main/kotlin/com/depanalyzer/repository/NvdClient.kt.
- Actualizacion guiada:
src/main/kotlin/com/depanalyzer/cli/UpdateCommand.kt.
- Sitios oficiales: Gradle, Maven, Kotlin, Sonatype OSS Index y NIST NVD.

1.5 Vision General

DepAnalyzer se visiona como una herramienta academica aplicada que permite evaluar rapidamente el estado de dependencias de proyectos multi-ecosistema. La propuesta de valor central es reducir incertidumbre tecnica al consolidar, en una sola ejecucion CLI, la informacion de:

- Dependencias declaradas.
- Versiones potencialmente desactualizadas.
- Vulnerabilidades directas y transitivas.

Desde la perspectiva de calidad de software, el sistema aporta deteccion temprana de riesgos, soporte para decisiones de mantenimiento y evidencia objetiva para auditorias internas de proyecto.

2. Posicionamiento

2.1 Oportunidad de negocio

En proyectos modernos, el uso intensivo de librerías de terceros incrementa la probabilidad de incorporar componentes desactualizados o vulnerables. La revisión manual de estos componentes suele ser costosa y no escalable, especialmente en entornos de aprendizaje con recursos limitados.

La oportunidad del proyecto se ubica en proveer una solución local y de bajo costo de adopción para:

- Cursos universitarios orientados a calidad, pruebas y seguridad.
- Equipos pequeños que requieren diagnósticos rápidos.
- Flujos de integración continua que demandan salidas estructuradas.

Propuesta de valor resumida:

- Implementación ligera basada en CLI.
- Cobertura de Maven y Gradle en una sola herramienta.
- Reporte directo y JSON para consumo humano y automatizado.

2.2 Definición del problema

El problema principal es la baja visibilidad del estado real de dependencias en proyectos Maven, Gradle, npm y Python, lo que genera dos consecuencias directas:

1. **Riesgo de seguridad:** inclusión de componentes con CVEs conocidos.
2. **Deuda técnica:** permanencia de versiones obsoletas con potencial impacto en estabilidad y mantenimiento.

Causas identificadas:

- Múltiples formatos de configuración en el ecosistema Java.
- Complejidad de dependencias transitivas.
- Falta de una rutina sistemática de verificación en etapas tempranas.

Efecto esperado con DepAnalyzer:

- Reducir tiempo de diagnóstico.
- Mejorar la priorización de acciones correctivas.
- Fortalecer prácticas de calidad y seguridad en el ciclo de desarrollo.

3. Descripción de los interesados y usuarios

3.1 Resumen de los interesados

| Interesado | Rol principal | Interes | Criterio de éxito | |---|---|---|---| | Estudiantes desarrolladores | Implementar y mantener la herramienta | Cumplir objetivos académicos y técnicos | Entregables funcionales y documentados | | Docente del curso | Supervisar y evaluar el proyecto | Evidencia de calidad del proceso y producto | Coherencia entre documento, código y pruebas | | Universidad | Institución formadora | Promover proyectos aplicados | Producto reutilizable en contextos educativos | | Comunidad técnica | Usuario potencial | Acceder a una herramienta útil y abierta | Facilidad de uso y resultados confiables |

3.2 Resumen de los usuarios

El usuario objetivo tiene perfil técnico y conoce el uso básico de terminal. Se identifican tres grupos:

- **Usuario académico:** ejecuta análisis para validar tareas de calidad.
- **Usuario desarrollador:** evalúa dependencias antes de liberar cambios.
- **Usuario mantenedor:** usa reportes para plan de actualizaciones.

3.3 Entorno de usuario

- Sistema operativo: Windows, Linux o macOS.
- Requisito de ejecución: JDK 25 o superior.
- Entorno de trabajo: terminal local y, opcionalmente, pipeline CI.
- Formato de resultados: texto en consola o JSON.

Escenarios representativos de uso:

- **Escenario A (manual):** usuario ejecuta `analyze` sobre la raíz del proyecto y revisa hallazgos en consola.
- **Escenario B (automatizado):** usuario ejecuta `analyze --output json` y procesa el resultado en otro paso.
- **Escenario C (entorno sin color):** usuario ejecuta `analyze --no-color` para logs limpios.

3.4 Perfiles de los interesados

- **Docente evaluador:** enfoque en trazabilidad, calidad documental y consistencia metodológica.
- **Equipo desarrollador:** enfoque en correctitud funcional, mantenibilidad y cobertura de pruebas.
- **Institución académica:** enfoque en impacto formativo y continuidad del proyecto.

3.5 Perfiles de los usuarios

- **Usuario técnico básico:** requiere comandos simples y salida interpretable.
- **Usuario técnico intermedio:** requiere salida JSON y parámetros de control.
- **Usuario técnico avanzado:** integra resultados en flujo CI o scripts.

3.6 Necesidades de los interesados y usuarios

| Necesidad | Tipo | Respuesta del sistema | |---|---|---| | Detectar componentes con riesgo de seguridad | Funcional | Consulta de CVEs y clasificacion en reporte | | Identificar desactualizaciones de dependencias | Funcional | Comparacion entre version actual y version mas reciente | | Obtener reporte legible y accionable | Usabilidad | Renderizado en consola con resumen por categoria | | Integrar salida a automatizaciones | Interoperabilidad | Exportacion JSON (--output json) | | Mantener facilidad de adopcion | Operativa | Ejecucion local por CLI sin infraestructura compleja |

4. Vista General del Producto

4.1 Perspectiva del producto

DepAnalyzer es una herramienta complementaria a los ecosistemas Maven, Gradle, npm y Python. Su rol no es reemplazar el sistema de build, sino inspeccionar configuraciones existentes y producir informacion de apoyo para la toma de decisiones.

Flujo funcional general:

1. Deteccion del tipo de proyecto.
2. Parseo de dependencias y repositorios.
3. Consulta de versiones y vulnerabilidades.
4. Clasificacion de hallazgos.
5. Emision de reporte en consola o JSON.

4.2 Resumen de capacidades

ID	Capacidad	Estado	Evidencia tecnica
CAP-01	Deteccion automatica de tipo de proyecto	Implementado	<code>ProjectDetector.detect</code>
CAP-02	Parseo Maven/Gradle, npm y Python	Implementado	<code>Modulos parser/*</code>
CAP-03	Lectura de repositorios declarados	Implementado	<code>Parsers y ProjectRepository</code>
CAP-04	Deteccion de dependencias desactualizadas	Implementado	<code>ProjectAnalyzer.findLatestVersion</code>
CAP-05	Consulta de CVEs con OSS Index	Implementado	<code>OssIndexClient.getVulnerabilities</code>
CAP-06	Consulta de CVEs con NVD	Implementado	<code>NvdClient, VulnerabilityMerger, flag --nvd</code>
CAP-07	Clasificacion directas/transitivas	Implementado	<code>ProjectAnalyzer.classifyVulnerabilities</code>
CAP-08	Salida JSON estructurada	Implementado	<code>ReportGenerator.toJson</code>
CAP-09	Salida consola sin color opcional	Implementado	<code>--no-color en CLI</code>
CAP-10	Actualizacion guiada con backup y dry-run	Implementado	<code>UpdateCommand, BuildFileBackup, --dry-run</code>

4.3 Suposiciones y dependencias

Suposiciones de operacion:

- El proyecto analizado posee estructura raiz valida.
- Los archivos de build se encuentran en ubicaciones estandar.
- Existe conectividad de red para consultas a servicios externos.

Dependencias tecnicas principales:

- Kotlin JVM y Gradle Kotlin DSL.
- Clikt y Mordant para experiencia CLI.
- OkHttp para consumo HTTP.
- Jackson para serializacion JSON.
- maven-model para parseo XML de Maven.

Dependencias externas de informacion:

- OSS Index para vulnerabilidades.
- Repositorios de artefactos para metadata de versiones.

4.4 Costos y precios

El proyecto tiene orientacion academica y de codigo abierto. No se establece un precio de comercializacion para esta fase. Segun FD01, el costo directo de desarrollo es bajo y concentrado en operacion basica, mientras que el stack tecnologico utiliza herramientas de acceso gratuito para fines educativos.

Modelo de adopcion esperado:

- **Costo de licencia:** no definido para uso academico interno.
- **Costo de instalacion:** bajo, asociado a tener JDK y entorno Gradle.
- **Costo de operacion:** bajo, dependiendo de conectividad para consultas externas.

4.5 Licenciamiento e instalacion

La instalacion se basa en build local y distribucion de CLI:

- Construcccion de distribucion con Gradle.
- Ejecucion mediante scripts generados (depanalyzer / depanalyzer.bat).
- Uso del comando analyze con ruta del proyecto.

Comandos de referencia (ejemplo):

```
./gradlew installDist  
./build/install/depanalyzer/bin/depanalyzer.bat analyze .  
./build/install/depanalyzer/bin/depanalyzer.bat analyze . --output json
```

En caso de requerir autenticacion para OSS Index, el token puede proveerse por opcion CLI (`--oss-token`) o variable de entorno (`OSS_INDEX_TOKEN`).

5. Características del producto

5.1 Requisitos funcionales principales

ID	Requisito funcional	Descripcion resumida
RF-01	Detectar tipo de proyecto	El sistema identifica automaticamente Maven, Gradle Groovy o Gradle Kotlin
RF-02	Leer dependencias declaradas	El sistema extrae coordenadas de dependencias por parser correspondiente
RF-03	Obtener version mas reciente	El sistema consulta metadata en repositorios configurados
RF-04	Detectar vulnerabilidades	El sistema consulta OSS Index para coordenadas detectadas
RF-05	Consultar CVEs con NVD	El sistema permite usar datos oficiales de NVD mediante --nvd
RF-06	Clasificar vulnerabilidades	El sistema diferencia hallazgos directos y transitivos
RF-07	Mostrar salida en consola	El sistema presenta resumen y detalle para lectura humana
RF-08	Exportar salida JSON	El sistema serializa el reporte para integracion automatizada
RF-09	Soportar token por CLI/entorno	El sistema recibe credencial por opcion o variable de entorno
RF-10	Actualizar dependencias de forma guiada	El sistema permite seleccionar cambios, confirmar y aplicar sobre build files
RF-11	Simular cambios sin modificar archivos	El sistema soporta --dry-run para previsualizar actualizaciones

5.2 Requisitos no funcionales asociados

- **RNF-01 (Portabilidad):** ejecucion en sistemas operativos comunes sobre JVM.
- **RNF-02 (Usabilidad tecnica):** sintaxis de comandos simple y ayuda clara.
- **RNF-03 (Mantenibilidad):** separacion modular entre parser, repositorio, core y reportes.
- **RNF-04 (Confiabilidad):** manejo de errores durante ejecucion y respuesta controlada.
- **RNF-05 (Interoperabilidad):** formato JSON estable para consumo por scripts.

5.3 Escenarios de uso prioritarios

1. **Revision previa a entrega academica:** analizar proyecto para verificar estado de dependencias.
2. **Control previo a merge:** ejecutar analisis en rama antes de integrar cambios.
3. **Generacion de evidencia:** producir JSON para anexos o reportes de calidad.

6. Restricciones

6.1 Restricciones tecnicas

- Requiere JDK compatible y ejecucion en entorno JVM.
- Requiere acceso de red para consulta de CVEs y metadata remota.
- El analisis depende de la estructura y sintaxis de los archivos de build.

6.2 Restricciones funcionales

- No realiza cambios automaticos sin confirmacion explicita del usuario.
- No incluye interfaz grafica dedicada en la version actual.
- No reemplaza herramientas de seguridad empresariales con cobertura ampliada.

6.3 Restricciones de proyecto academico

- Alcance acotado al periodo del curso.
- Priorizacion de entregables verificables sobre funcionalidades accesorias.
- Evolucion incremental sujeta a tiempo disponible y validacion docente.

7. Rangos de Calidad

Atributo	Indicador	Meta objetivo	Metodo de verificacion
Correctitud funcional	Dependencias correctamente extraidas	$\geq 90\%$ en casos de prueba definidos	Pruebas unitarias de parser
Confiabilidad	Ejecuciones sin error en entradas validas	$\geq 95\%$ de ejecuciones exitosas	Pruebas de integracion basica
Rendimiento	Tiempo de analisis en proyecto pequeno/mediano	≤ 30 s en entorno local de referencia	Medicion por corrida controlada
Usabilidad tecnica	Comprension del reporte por usuario tecnico	$\geq 80\%$ de comprension en evaluacion interna	Revision de usuarios del curso
Interoperabilidad	JSON valido y parseable	100% de reportes JSON validos	Validacion de estructura de salida
Mantenibilidad	Modulos con pruebas asociadas	Cobertura en componentes criticos	Revision de suite de test

Criterios de aceptacion global:

- El sistema se considera aceptable si cumple al menos los umbrales de correctitud, confiabilidad e interoperabilidad.
- Rendimiento y usabilidad se consideran metas de mejora continua si el entorno de prueba presenta variaciones.

8. Precedencia y Prioridad

Nivel	Elemento	Tipo	Justificacion academica
Alta	Deteccion de proyecto y parseo de dependencias	Nucleo funcional	Sin este bloque no existe analisis util
Alta	Deteccion de vulnerabilidades (CVE)	Seguridad	Aporta valor principal del proyecto
Alta	Reporte claro en consola	Usabilidad	Permite interpretacion inmediata de resultados
Media	Exportacion JSON	Integracion	Favorece automatizacion y trazabilidad
Media	Parametros de ejecucion (--no-color, token)	Operativa	Mejora adaptabilidad a entornos diversos
Baja	Caracteristicas avanzadas adicionales	Evolutivo	Puede incorporarse en iteraciones futuras

Criterio de precedencia aplicado:

Se priorizan funcionalidades que impactan directamente la deteccion de riesgo y la evidencia de calidad. Las mejoras de experiencia o extensiones quedan subordinadas al cumplimiento del nucleo funcional.

9. Otros requerimientos del producto

9.1 Estándares legales

- Respetar terminos de uso de APIs y repositorios consultados.
- Mantener atribucion de bibliotecas de terceros utilizadas.
- Preservar buenas practicas de propiedad intelectual en contexto academico.
- Evitar inclusion de datos sensibles en reportes compartidos.

9.2 Estándares de comunicacion

- Reportar resultados con lenguaje tecnico claro y verificable.
- Documentar cambios relevantes mediante control de versiones del repositorio.
- Facilitar trazabilidad entre requisitos, implementacion y pruebas.

9.3 Estándares de cumplimiento de la plataforma

- Cumplir convenciones de proyecto Kotlin/Gradle.
- Mantener compatibilidad con ejecucion por CLI en JVM.
- Preservar estructura modular del codigo (cli, core, parser, repository, report).
- Mantener salida JSON estable para integracion externa.

9.4 Estándares de calidad y seguridad

- Ejecutar pruebas unitarias y de integracion para componentes criticos.
- Manejar errores de red y excepciones de parseo con mensajes controlados.
- No realizar modificaciones destructivas en el proyecto analizado.
- Priorizar transparencia del reporte (detalle de dependencia y severidad).

CONCLUSIONES

1. La vision del proyecto queda definida con mayor precision tecnica y academica, delimitando claramente su alcance y sus fronteras.
2. DepAnalyzer responde a una necesidad concreta de control de dependencias y vulnerabilidades en proyectos multi-ecosistema.
3. La estructura funcional implementada en CLI, junto con salida en consola y JSON, habilita tanto uso manual como semiautomatizado.
4. El documento establece criterios medibles de calidad que permiten evaluar el avance del producto de forma objetiva.
5. La priorizacion propuesta favorece la entrega de valor real en el nucleo del sistema y ordena su evolucion futura.

RECOMENDACIONES

1. Mantener sincronizados docs/FD01-Informe-Factibilidad.md, docs/FD02-Informe-Vision.md y README.md en cada iteracion.
2. Incorporar una matriz de trazabilidad formal (requisito -> modulo -> prueba) como anexo del proyecto.
3. Definir una linea base de medicion para los indicadores de calidad (tiempo, tasa de exito y cobertura).
4. Fortalecer pruebas para escenarios de error de red y proyectos con configuraciones no estandar.
5. Evaluar, en fases posteriores, funciones de apoyo a remediacion guiada y reportes comparativos entre ejecuciones.

BIBLIOGRAFIA

1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*.
2. Sommerville, I. (2016). *Software Engineering*.
3. ISO/IEC 25010:2011. *Systems and software quality models*.
4. Sonatype. (2026). *OSS Index Documentation*.
5. Apache Software Foundation. (2026). *Maven Documentation*.
6. Gradle Inc. (2026). *Gradle User Manual*.
7. JetBrains. (2026). *Kotlin Documentation*.

WEBGRAFIA

- Repositorio del proyecto: README.md, build.gradle.kts.
- Código fuente CLI: src/main/kotlin/com/depanalyzer/cli/DepAnalyzerCli.kt.
- Código fuente core: src/main/kotlin/com/depanalyzer/core/ProjectAnalyzer.kt.
- Código fuente reportes:
src/main/kotlin/com/depanalyzer/report/DependencyReport.kt.
- <https://ossindex.sonatype.org/>
- <https://docs.gradle.org/>
- <https://maven.apache.org/>
- <https://kotlinlang.org/docs/home.html>

10. Wiki y Roadmap de Versiones

La fuente de la Wiki se mantiene en `wiki/` y se sincroniza mediante `wiki.yml`.

Versión	Estado	Objetivo	Entregables
v1.x	Completada	Analizador Maven/Gradle	Parsers, versiones, OSS Index, árbol transitivo, CLI y JSON
v2.x	Actual	Solución multi-ecosistema auditable	npm/Python, NVD, TUI, actualización, binarios, MCP y calidad automatizada
v3.x	Planificada	Adopción y operación continua	GitHub Action, imagen OCI, SBOM, firma, caché e histórico

10.1 Criterios de cierre

Cada versión requiere pruebas exitosas, cobertura mínima, reportes de seguridad, documentación actualizada, tag y release. La versión actual corresponde a la segunda línea principal; v3.x permanece como planificación.